pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Z-Scores and Percentile Ranks

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: make two phrases exact

The Desk Lead's Note

This morning's note says the 10-year is "near the top of its one-year range" and 2s10s is "about average." Make both exact for 2 October 2026: a z-score and a percentile rank over the last 252 market days, each defined in writing and tied to Excel, both added to "marketdata" with tests that prove they use no future data. Then write the two sentences for the note: where each sits, never where it goes.

Every number today comes from the Treasury file, so there is no key and no live cell.

```
Work folder pfi_course3_08_k4w7q2xr: 3 data files, Course 2's bondmath.py, marketdata.py as notebook 07 left it, 7 commits, tagged week1
.........................   [100%]
29 passed in 0.50s
```

Imports and the Two Series

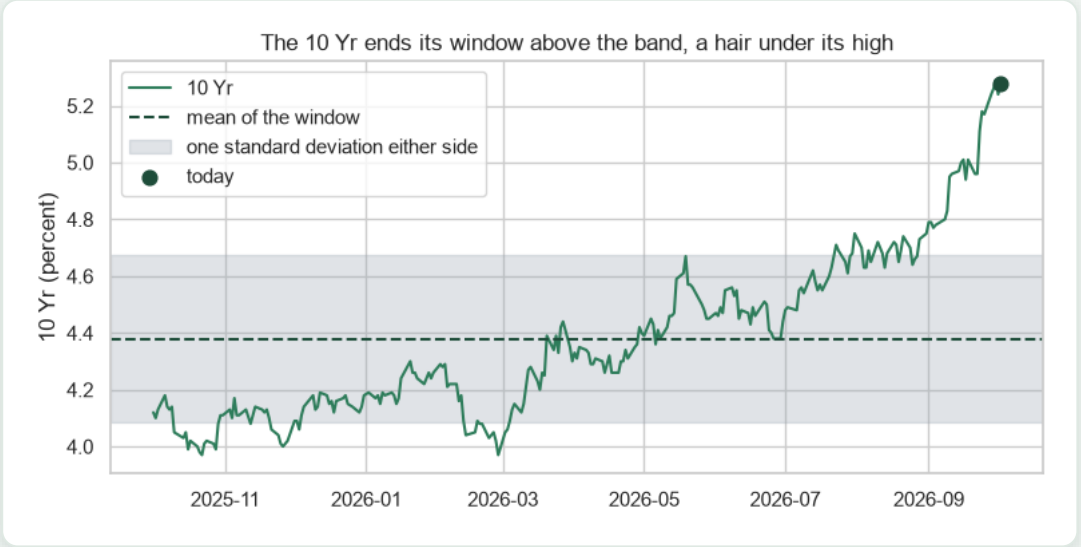
```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
ten_year = treasury['10 Yr']
slope_2s10s = (treasury['10 Yr'] - treasury['2 Yr']) * 100
print(f'{len(treasury):,} market days; on {treasury.index[-1]:%Y-%m-%d} the 10 Yr is '
      f'{ten_year.iloc[-1]:.2f} percent and 2s10s {slope_2s10s.iloc[-1]:.1f} bp')
2,940 market days; on 2026-10-02 the 10 Yr is 5.28 percent and 2s10s 45.0 bp
```

Notebook 07 measured how much a series moves; today asks where its level sits.

8.1 Where does today sit in its own history?

"Near the top of its one-year range" needs a range first.






```
window = ten_year.iloc[-252:]
today = window.iloc[-1]
print(f'{window.index[0]:%Y-%m-%d} to {window.index[-1]:%Y-%m-%d}: today {today:.2f} percent')
print(f'low {window.min():.2f} on {window.idxmin():%Y-%m-%d}, '
      f'high {window.max():.2f} on {window.idxmax():%Y-%m-%d}, mean {window.mean():.4f}')
2025-10-01 to 2026-10-02: today 5.28 percent
low 3.97 on 2025-10-22, high 5.29 on 2026-09-30, mean 4.3798
```

Three ways to say where

```
distance_bp = (today - window.mean()) * 100
std_bp = window.std() * 100
print(f'{distance_bp:.1f} bp above the mean; one standard deviation is {std_bp:.1f} bp: '
      f'{distance_bp / std_bp:.4f} of them')
90.0 bp above the mean; one standard deviation is 29.3 bp: 3.0752 of them
```

The distance in standard deviations (sd for short) is the z-score: 3.0752.

Excel side by side. The sheet holds dates in A, the 2 Yr in B, the 10 Yr in C and 2s10s in D, from 2025-09-30 in row 2 to 2026-10-02 in row 254. The window is C3:C254.

Job	Python	Excel	Result
Low and high	`window.min(), window.max()`	`=MIN(C3:C254), =MAX(C3:C254)`	3.97, 5.29
The high's date	`window.idxmax()`	`=XLOOKUP(MAX(C3:C254),C3:C254,A3:A254)`	46295, 2026-09-30 as a date
Distance from the mean, bp	`distance_bp`	`=(C254-AVERAGE(C3:C254))*100`	90.02381
One standard deviation, bp	`std_bp`	`=STDEV.S(C3:C254)*100`	29.2738
Days at or below today	`at_or_below`	`=COUNTIF(C3:C254,"<="&C254)`	

"<="&C254 joins the text "<=" to today's value, so "COUNTIF" counts the cells at or below today.

8.2 The z-score, written down

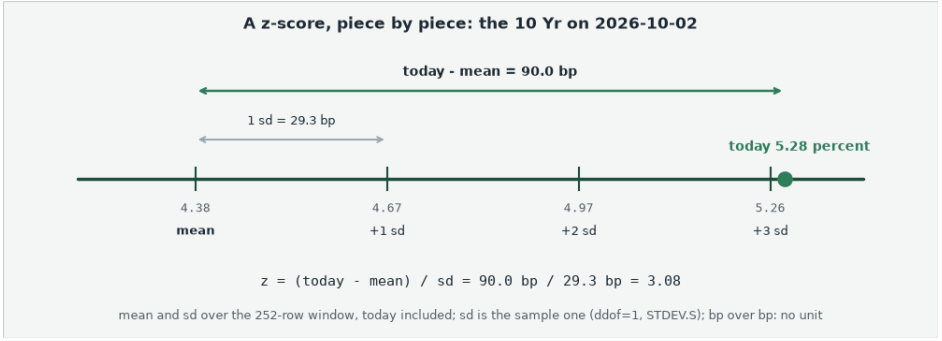
```
%%writefile -a marketdata.py
def rolling_zscore(series: pd.Series, window: int = 252) -> pd.Series:
    """Where each value sits in its own trailing window, in standard deviations from the window's mean.
    Units: none (a number of standard deviations). Convention: z_t = (x_t - mean) / sample standard deviation,
    both over the trailing window that includes x_t (window rows, min_periods = window, ddof=1). NaN where the
    window's standard deviation is 0 (a flat, stale stretch). Describes position only; it is not a signal.
    Excel: =STANDARDIZE(B253,AVERAGE(B2:B253),STDEV.S(B2:B253)) filled down
    Raises ValueError if window is below 2, the series has missing values, or its index is not unique
    ascending dates.
    """
    _check_series(series, "series")
    if window < 2:
        raise ValueError(f"window must be at least 2 rows, not {window}")
    mean = series.rolling(window, min_periods=window).mean()
    std = series.rolling(window, min_periods=window).std(ddof=1)
    std = std.where(std != 0)
    return (series - mean) / std
```

8.1 found 3.0752 by hand; the module should find it on every date.

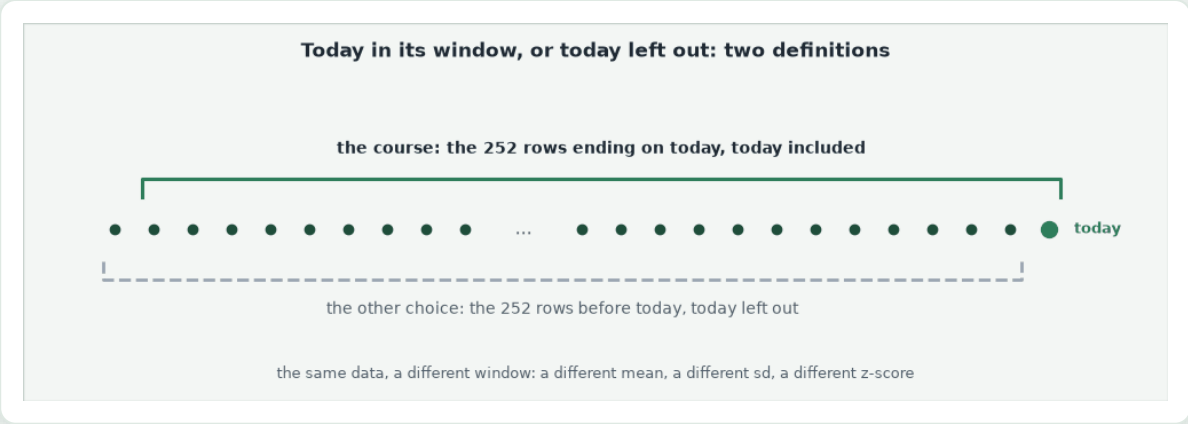
Same Number as by Hand

```
rolling_zscore today: 3.075234; by hand: 3.075234
251 NaN before the first full window
```

251 empty values, then the first z-score on row 252, notebook 07's rule.



Today in or today out, and a flat window



The course puts today in its own window. The other choice compares today with the 252 rows before it.

```
before = ten_year.iloc[-253:-1]
z_left_out = (today - before.mean()) / before.std()
print(f'today in: {z_ten_year.iloc[-1]:.4f}; today left out: {z_left_out:.4f}')
today in: 3.0752; today left out: 3.1470
```

Neither choice is wrong; a number with no definition beside it is. The course includes today, as "STANDARDIZE" does over a range that ends on today's cell.

A Flat Window

Q. What is the z-score of a stretch that does not move?

```
stale = pd.Series([5.0, 5.0, 5.0, 5.0, 5.1, 5.0], index=pd.bdate_range('2026-09-01', periods=6))
flat = pd.DataFrame({'value': stale, 'z, 3 rows': md.rolling_zscore(stale, window=3)})
flat.round(2)
```

	value	z, 3 rows
2026-09-01	5.0	NaN
2026-09-02	5.0	NaN
2026-09-03	5.0	NaN
2026-09-04	5.0	NaN
2026-09-07	5.1	1.15
2026-09-08	5.0	-0.58

The first two rows are partial windows.

Three tests, and STANDARDIZE

The z-score against Excel's "STANDARDIZE" on all 190 rows of 2026 for both series, the flat window, and no lookahead with notebook 07's "assert_no_lookahead" and cut dates. "ten_year_and_2s10s" builds both series as the Excel sheet does.

```
%%writefile -a test_marketdata.py
def ten_year_and_2s10s() -> dict[str, pd.Series]:
    curve = read_treasury()
    return {"10y": curve["10 Yr"], "2s10s": (curve["10 Yr"] - curve["2 Yr"]) * 100}
def test_zscore_matches_excel_standardize():
    reference = read_reference()
    for name, series in ten_year_and_2s10s().items():
        z = md.rolling_zscore(series, window=252)
        for day, row in reference.iterrows():
            assert z.loc[day] == pytest.approx(row[f"zscore252_{name}"], abs=1e-9), (name, day)
def test_zscore_nan_when_flat():
    dates = pd.bdate_range("2026-09-01", periods=6)
    stale = pd.Series([5.0, 5.0, 5.0, 5.0, 5.1, 5.0], index=dates)
    z = md.rolling_zscore(stale, window=3)
    assert z.iloc[:4].isna().all()
    assert z.iloc[4:].notna().all()
def test_zscore_no_lookahead():
    for series in ten_year_and_2s10s().values():
        assert_no_lookahead(lambda s: md.rolling_zscore(s, window=252), series, CUT_DATES)
```

```
................................   [100%]
32 passed in 0.50s
```

"rolling_zscore" is Excel's "STANDARDIZE" on all 190 rows of 2026 for both series, and no future row reaches it.

Excel side by side. Filled down from row 253, each row's formula gets its own trailing window, as "rolling(252)" does.

Job	Python	Excel	Result
Z-score, today in	`md.rolling_zscore(ten_year)`	`=STANDARDIZE(C254,AVERAGE(C3:C254),STDEV.S(C3:C254))`	3.075234
Today left out	`z_left_out`	`=STANDARDIZE(C254,AVERAGE(C2:C253),STDEV.S(C2:C253))`	3.147044
A flat window, 5, 5, 5	`md.rolling_zscore(stale, window=3): NaN`	`=STANDARDIZE(A3,AVERAGE(A1:A3),STDEV.S(A1:A3))`	#NUM!
An empty value instead	`NaN`	`=IFERROR(STANDARDIZE(5,AVERAGE(I1:I3),STDEV.S(I1:I3)),NA())`	#N/A

8.3 The percentile rank, and ties

The percentile rank is 100 x the window's values at or below today / the values in the window. Today counts itself, so it runs from above 0 up to 100.

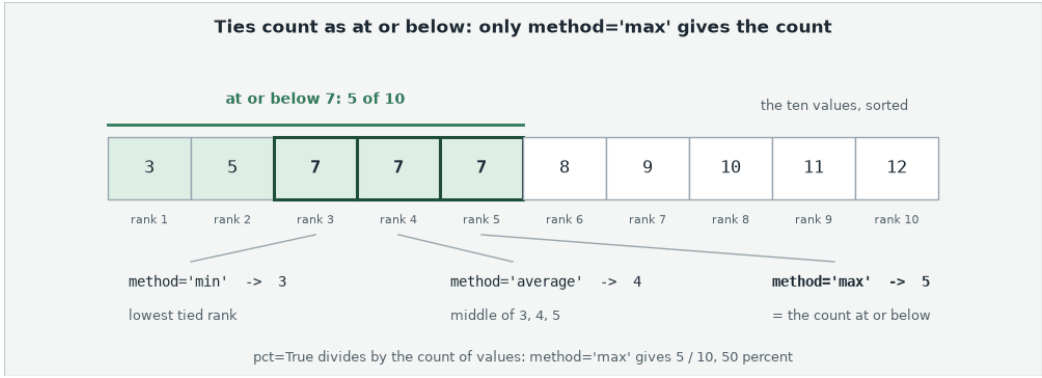
```
print(window.nunique(), 'different values in', len(window), 'days of the 10 Yr')
94 different values in 252 days of the 10 Yr
```

Yields are quoted to two decimals, so many days tie.

```
probe = pd.Series([5, 7, 7, 9, 11, 12, 3, 7, 10, 8],
                  index=pd.bdate_range('2026-09-14', periods=10), dtype=float)
print(f'at or below 7: {(probe <= 7).sum()} of 10; at or below 8: {(probe <= 8).sum()} of 10')
at or below 7: 5 of 10; at or below 8: 6 of 10
```

Q. pandas' "rank" gives each value its place, 1 for the smallest. What do the three 7s get?

```
for method in ['min', 'average', 'max']:
    print(f"method='{method}': the 7s rank {probe.rank(method=method).iloc[1]:.0f}")
method='min': the 7s rank 3
method='average': the 7s rank 4
method='max': the 7s rank 5
```



The tie rule in Excel, and two 32s that are not equal

Only "max" gives each 7 the count at or below it. "pct=True" divides that count by the number of values, so "rank(pct=True, method='max')" is the percentile rank as a share.

Excel Side by Side

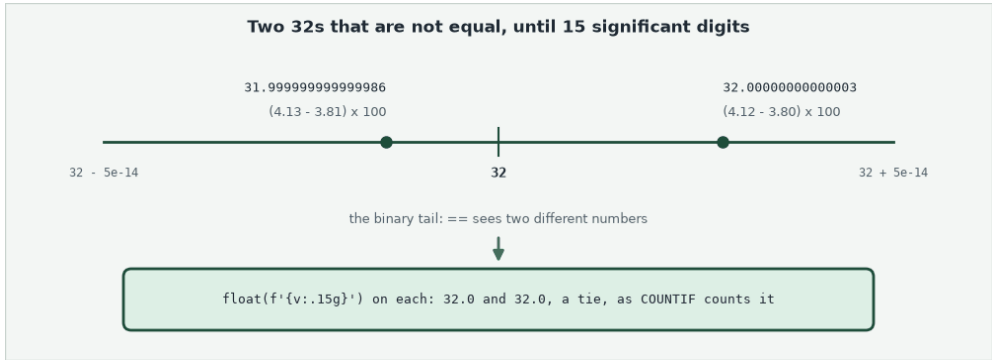
Job	Python	Excel	Result
Lowest tied rank	`probe.rank(method='min')`	`=RANK.EQ(7,A1:A10,1)`	3
Average tied rank	`probe.rank(method='average')`	`=RANK.AVG(7,A1:A10,1)`	4
At or below 7, percent	`probe.rank(pct=True, method='max') * 100`	`=COUNTIF(A1:A10,"<="&7)/COUNT(A1:A10)*100`	50

Two 32 bp Slopes

```
slope_a = (4.12 - 3.80) * 100
slope_b = (4.13 - 3.81) * 100
print(f'{slope_a!r} and {slope_b!r}: equal? {slope_a == slope_b}')
print(float(f'{slope_a:.15g}'), float(f'{slope_b:.15g}'))
32.00000000000003 and 31.999999999999986: equal? False
32.0 32.0
```

Excel's Answers for 2026

```
reference = pd.read_csv('course3_excel_reference.csv', parse_dates=['date'], index_col='date',
                        float_precision='round_trip')
print(len(reference), f'rows, {reference.index[0]:%Y-%m-%d} to {reference.index[-1]:%Y-%m-%d}')
190 rows, 2026-01-02 to 2026-10-02
```



In a sheet with the two and 31 in A1:A3, "=COUNTIF(A1:A3,"<="&A3)/COUNT(A1:A3)*100" returns 100: Excel counts the two as a tie.

8.4 The percentile rank in the module

Round every value to 15 significant digits first, as the two 32s showed, then rank as plain pandas did. "rolling_percentile_rank" does the rounding with ".map(lambda ...)", Course 1's ".map" and "lambda".

```
%%writefile -a marketdata.py
def rolling_percentile_rank(series: pd.Series, window: int = 252) -> pd.Series:
    """The share of the trailing window at or below each value, in percent.
    Units: percent, above 0 up to 100. Convention: 100 x (number of window values at or below x_t) / window,
    over the trailing window that includes x_t (min_periods = window). Ties count as at or below.
    Values are compared at 15 significant digits, as Excel's COUNTIF compares them: in binary arithmetic
    (4.13 - 3.81) x 100 is 31.999999999999986 and (4.12 - 3.80) x 100 is 32.00000000000003, and the two count
    as a tie at 32 bp.
    Excel: =COUNTIF(B2:B253,"<="&B253)/COUNT(B2:B253)*100 filled down.
    Raises ValueError if window is below 2, the series has missing values, or its index is not unique
    ascending dates.
    """
    _check_series(series, "series")
    if window < 2:
        raise ValueError(f"window must be at least 2 rows, not {window}")
    rounded = series.map(lambda value: float(f"{value:.15g}"))
    return rounded.rolling(window, min_periods=window).rank(pct=True, method="max") * 100
```

The Function Against COUNTIF

```
probe set, last value 8: 60 percent
10y: differs from COUNTIF on 0 of 190 rows
2s10s: differs from COUNTIF on 0 of 190 rows
```

Three tests, and COUNTIF

"COUNTIF" on all 190 rows, the two 32s as a tie, and no lookahead.

```
%%writefile -a test_marketdata.py
def test_percentile_matches_excel_countif():
    reference = read_reference()
    for name, series in ten_year_and_2s10s().items():
        rank = md.rolling_percentile_rank(series, window=252)
        for day, row in reference.iterrows():
            assert rank.loc[day] == pytest.approx(row[f"pctrank252_{name}"], abs=1e-12), (name, day)
def test_percentile_ties_count_as_at_or_below():
    dates = pd.bdate_range("2026-09-01", periods=4)
    values = pd.Series([7.0, 9.0, 7.0, 7.0], index=dates)
    rank = md.rolling_percentile_rank(values, window=4)
    assert rank.iloc[-1] == pytest.approx(75.0, abs=1e-12)
    noisy = pd.Series([(4.12 - 3.80) * 100, 31.0, (4.13 - 3.81) * 100], index=dates[:3])
    assert noisy.iloc[0] > 32 > noisy.iloc[2]
    assert md.rolling_percentile_rank(noisy, window=3).iloc[-1] == pytest.approx(100.0, abs=1e-12)
def test_percentile_no_lookahead():
    for series in ten_year_and_2s10s().values():
        assert_no_lookahead(lambda s: md.rolling_percentile_rank(s, window=252), series, CUT_DATES)
```

```
.................................   [100%]
35 passed in 0.50s
```

Excel side by side. "COUNT" counts the numbers in the window, 252.

Job	Python	Excel	Result
Percentile rank, 2s10s	`md.rolling_percentile_rank(slope_2s10s)`	`=COUNTIF(D3:D254,"<="&D254)/COUNT(D3:D254)*100`	28.174603
The two 32s and 31	`md.rolling_percentile_rank(noisy, window=3)`	`=COUNTIF(A1:A3,"<="&A3)/COUNT(A1:A3)*100`	100

The same tail can split a tie wherever computed numbers are compared for equality, in "==", a rank, or a merge.

8.5 Excel's PERCENTRANK.INC

Excel also has a function named for the job, and an assistant asked for "the percentile rank in Excel" may suggest it.

On Every Reference Row

```
for name, series in [('10y', ten_year), ('2s10s', slope_2s10s)]:
    inc = pd.Series([percentrank_inc(series.loc[:day].iloc[-252:], series.loc[day])
                     for day in reference.index], index=reference.index)
    same = (inc == reference[f'percentrank_inc252_{name}']).sum()
    far = (inc * 100 - reference[f'pctrank252_{name}']).abs().max()
    print(f'{name}: PERCENTRANK.INC on {same} of {len(inc)} rows; '
          f'up to {far:.2f} points from COUNTIF')
```

Excel Side by Side

Job	Python	Excel
The ten values, 7	`percentrank_inc(probe, 7)`	`=PERCENTRANK.INC(A1:A10,7)`
The ten values, 8	`percentrank_inc(probe, 8)`	`=PERCENTRANK.INC(A1:A10,A10)`
The 10 Yr today	`percentrank_inc(window, today)`	`=PERCENTRANK.INC(C3:C254,C254)`
2s10s today	`percentrank_inc(slope_window, slope_today)`	`=PERCENTRANK.INC(D3:D254,D254)`

8.6 Both measures, every day of 2026

The note asked about one day; the same two numbers exist for every day of 2026.

```
measures = pd.DataFrame({
    '10y_pct': ten_year, '10y_z': z_ten_year, '10y_rank': md.rolling_percentile_rank(ten_year),
    '2s10s_bp': slope_2s10s, '2s10s_z': md.rolling_zscore(slope_2s10s),
    '2s10s_rank': md.rolling_percentile_rank(slope_2s10s),
}).loc['2026']
print(len(measures), 'days of 2026')
190 days of 2026
```

Month Ends

	10y_pct	10y_z	10y_rank	2s10s_bp	2s10s_z	2s10s_rank
2026-01-30	4.26	-0.02	53.57	74.00	1.87	100.00
2026-02-27	3.97	-1.92	0.79	59.00	0.50	75.79
2026-03-31	4.30	0.49	70.63	51.00	-0.59	27.38
2026-04-30	4.40	1.20	86.51	52.00	-0.57	36.11
2026-05-29	4.45	1.43	91.67	47.00	-1.26	7.14
2026-06-30	4.44	1.26	87.30	30.00	-2.76	2.38
2026-07-31	4.75	2.69	100.00	47.00	-0.63	21.03
2026-08-31	4.75	2.10	100.00	41.00	-1.15	15.08
2026-09-30	5.29	3.25	100.00	41.00	-0.86	23.02

Against Excel, Every Row

```
10y: largest gap, z-score 1.9e-12; percentile rank 0.0e+00 points
2s10s: largest gap, z-score 7.3e-14; percentile rank 0.0e+00 points
```

Inside the course's 1e-9 for the z-score; equal for the rank.

The Range of Each

```
10y_z: -1.92 on 2026-02-27 to 3.25 on 2026-09-30
2s10s_z: -3.54 on 2026-06-18 to 1.97 on 2026-01-02
```

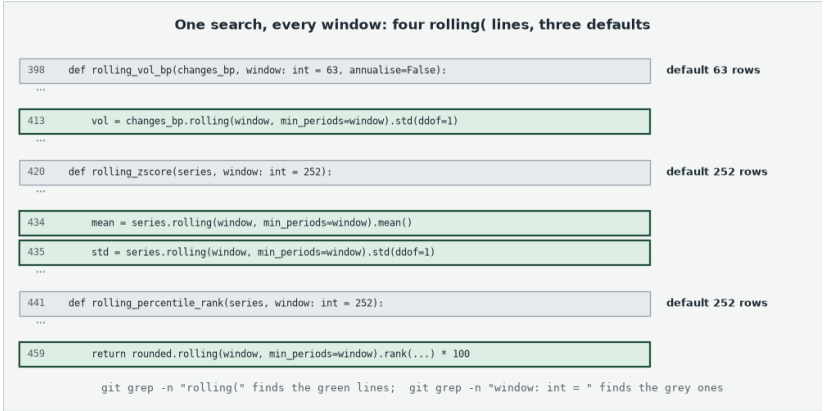
8.7 Commit, then find every window

The module gained two window functions today. Before anyone changes "the" default window, find every place one opens.

```
!git -C "{work}" diff --stat
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "Notebook 08: rolling_zscore and rolling_percentile_rank"
!git -C "{work}" log --oneline -3
 marketdata.py   | 42 ++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 54 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 96 insertions(+)
de7069f Notebook 08: rolling_zscore and rolling_percentile_rank
85a1fa6 Notebook 07: rolling_vol_bp and assert_no_lookahead
1f5838a Notebook 06: change_bp, return_pct, and change_since_bp
```

```
!git -C "{work}" grep -n "rolling(" -- marketdata.py
marketdata.py:413:    vol = changes_bp.rolling(window, min_periods=window).std(ddof=1)
marketdata.py:434:    mean = series.rolling(window, min_periods=window).mean()
marketdata.py:435:    std = series.rolling(window, min_periods=window).std(ddof=1)
marketdata.py:459:    return rounded.rolling(window, min_periods=window).rank(pct=True, method="max") * 100
```

```
!git -C "{work}" grep -n "window: int = " -- marketdata.py
marketdata.py:398:def rolling_vol_bp(changes_bp: pd.Series, window: int = 63, annualise: bool = False) -> pd.Series:
marketdata.py:420:def rolling_zscore(series: pd.Series, window: int = 252) -> pd.Series:
marketdata.py:441:def rolling_percentile_rank(series: pd.Series, window: int = 252) -> pd.Series:
```



"git grep" searches only the files Git tracks, so the data files and "bondmath.py" stay out of it.

Before changing a default window, "git grep" finds every place one opens: four lines in three functions, defaults of 63 and 252 rows.

8.8 Words that describe, words that forecast

A z-score of 3.08 invites a story; the course tells only the part the number holds.

```
On 2026-10-02 2s10s was 45.0 bp, 0.51 standard deviations below the mean of its last 252 market days, with 28.2 percent of those days at or below it.
```

Words that describe	Words that forecast or advise
3.08 standard deviations above the mean of its last 252 market days	stretched, extreme, overextended
the low and high of the window, with their dates	cheap, rich, attractive
up so many bp over the month (notebook 06)	due to revert, bound to come back
the z-score rose from one value to another	a signal, a trigger, a buy, a sell
the window is 252 rows and includes today	will, expect, should, likely

No threshold either: no line at a z-score of 2 or at the 90th percentile. A threshold turns a description into a decision rule, and a decision rule is a firm's, with a firm's reasons.

The note gets two sentences that say where each number sits and nothing about where it goes.

Bring it home, and what you can now do

Copy the work folder's two files into your own "my_bondmath" folder, test them there, and commit. The copy replaces your two files, so anything of your own must be in the work folder first. The tests read the Treasury file and Excel's reference file, in your folder since notebooks 04 and 06.

```
$work = "$env:TEMP\pfi_course3_08_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 08: rolling_zscore and rolling_percentile_rank"
git grep -n "rolling(" -- marketdata.py
```

pytest shows 35 passed, and "git grep" the same four lines. In Colab, download the two files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Say where today sits in its own window as a distance in bp, a z-score, and a percentile rank.

Compute a z-score over a trailing window that includes today, and show that leaving today out changes it.

Compute a percentile rank that counts ties as at or below, comparing at 15 significant digits as "COUNTIF" does.

Prove a window function uses no future data, and find every window in a module with "git grep".

Describe a level without forecasting it.

Predict first: the share below today, and today left out

Predict 1

Today the 10 Yr is 5.28 percent; over its last 252 market days it ran from 3.97 to 5.29. What share of those 252 days were at or below today, in percent? Write down a number.

```
at_or_below = (window <= today).sum()
print(f'{at_or_below} of {len(window)} days at or below today: '
      f'{at_or_below / len(window) * 100:.4f} percent')
```

Predict 2

Leaving today out moved the 10 Yr's z-score by 0.07. 2s10s's z-score today is -0.51. Leave today out of its window:

A) it barely moves, the same to two decimals

B) it moves about as much as the 10 Yr's

C) its sign flips

```
slope_today = slope_2s10s.iloc[-1]
slope_window, slope_before = slope_2s10s.iloc[-252:], slope_2s10s.iloc[-253:-1]
print(f'today in: {(slope_today - slope_window.mean()) / slope_window.std():.4f}')
print(f'today left out: {(slope_today - slope_before.mean()) / slope_before.std():.4f}')
```

Predict first: plain pandas, and PERCENTRANK.INC

Predict 3

Rank each window with plain pandas and compare with Excel's "COUNTIF" on 2s10s's 190 rows. They are:

A) equal on every row: the tail is too small to matter

B) different on a handful of rows

C) different on more than a third

```
for name, series in [('10y', ten_year), ('2s10s', slope_2s10s)]:
    plain = series.rolling(252, min_periods=252).rank(pct=True, method='max') * 100
    gap = (plain.loc[reference.index] - reference[f'pctrank252_{name}']).abs()
    print(f'{name}: differs from COUNTIF on {(gap > 1e-12).sum()} of {len(gap)} rows, '
          f'by up to {gap.max():.2f} points')
```

Predict 4

Of the ten values, 0.5 are at or below 7. "=PERCENTRANK.INC(A1:A10,7)" returns:

A) 0.5, the same

B) less than 0.5

C) more than 0.5

```
print('7:', percentrank_inc(probe, 7), '  8:', percentrank_inc(probe, 8))
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "mean = series.mean()" and "std = series.std(ddof=1)" read the whole file, so z-scores up to 2025-04-17 change when later rows arrive.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
2s10s	0.51 below, 28.2nd percentile
Defined and tied to Excel	STANDARDIZE and COUNTIF over COUNT, 190 rows each, 35 passed
No future data	both pass assert_no_lookahead
The sentences	two, both describing

Be proud: you checked two things most people never look at, today's place in its own window and a binary tail splitting a tie.

Next: Slopes and Butterflies (notebook 09), where these measures say where each slope and fly sits.